

The Conversational History-Taking Engine

A revised, implementation-grade architecture for Module A (dynamic history-taking) and Module B (document intelligence), built around a macro-FSM derived from Macleod's clinical history framework, with a toggleable AYUSH assessment layer and an explicit account of how STT, TTS, the LLM, and the dialogue state machine are wired together at runtime.

Supersedes: AI-generated research draft ("Generated by Kimi"), v1

Scope: Module A (Conversational History Engine) and Module B (Document Digitization) — with a critique of v1 and revised recommendations for Modules C/D where they intersect

Date: 27 August 2026

Status: Design reference for build planning — not yet clinically or regulatorily validated

0 Scope & how this document is organized

*This is not a rewrite from zero. The prior research draft got the macro-level system philosophy right — a three-layer ingestion/intelligence/integration pipeline, an edge/hybrid-cloud deployment split, and a genuinely solid FHIR R4 / ABDM resource-mapping table that should be kept largely as-is. What it does not do is answer the question that actually determines whether this product works: **how, mechanically, does the system decide what to ask next, in a way that is safe, auditable, and doesn't collapse into either a rigid questionnaire or an unpredictable free-running chatbot?** Sections 2-5 answer that in full. Section 7 does the same for document extraction. Section 6 covers the AYUSH toggle you asked for. Appendix A is a line-by-line critique of the prior draft's specific claims.*

Section	Content
1	Design philosophy — why a single end-to-end LLM conversation is the wrong architecture, and the three-tier alternative
2	The macro-FSM — states derived from Macleod's history-taking framework, transitions, interrupts
3	The meso layer — complaint-driven acronym templates (SOCRATES, and others) inside the HPI state
4	The micro layer — the actual dynamic-questioning mechanism: slot filling, opportunistic extraction, guarded generation
5	Runtime integration — how STT, TTS, the LLM, and the FSM are wired together turn by turn, and where each component physically runs
6	AYUSH mode as a gated configuration, not a patient-facing toggle — and a clinical correction to what belongs in it
7	Document extraction pipeline — what to keep from v1, what to simplify, and how it merges into the same patient record
8	The unified patient-record schema that ties every layer together
9	Red-flag / safety architecture — making the three layers in v1 actually resolve disagreements safely
10	Revised build sequence
Appendix A	Claim-by-claim critique of the v1 research draft

1 Design philosophy

The instinctive way to build "an AI that talks to the patient and takes a history" is one large LLM conversation loop: feed it the transcript so far, ask it to decide what to say next. This fails for a clinical-grade product for a specific, demonstrated reason, not a hypothetical one — when a general-purpose LLM is left to freely drive a diagnostic interview against a rule/graph-driven engine doing the same job, the free-running model reliably asks *fewer* questions and omits clinically relevant ones, because it optimizes for a plausible-sounding conversation, not for interview completeness. That is precisely the failure mode you cannot afford in an OPD intake tool a physician is going to trust.

CORE PRINCIPLE

Separate **what to ask** from **how to say it**. A deterministic, auditable policy (the FSM + ontology) decides the next piece of clinical information needed. The LLM's only jobs are (a) turning that decision into a natural, correctly-translated sentence, and (b) turning the patient's free-form answer back into structured data. The LLM never decides clinical branching on its own.

This single decision is what makes the rest of the document possible. It resolves the "how do I stop this from being a rigid form" worry (the LLM keeps it feeling conversational and lets patients answer multiple things at once) and the "how do I stop this from hallucinating or missing things" worry (the policy layer is a small, testable, hand-authored artifact you can unit-test against clinical vignettes, the same way the CareRoute-style adaptive symptom checkers are evaluated against standardized vignette sets before release).

Three tiers implement this, each solving a different-sized version of the same problem:

Tier	Answers	Mechanism	Why not one FSM for everything
Macro (§2)	Which <i>section</i> of the history are we in?	A literal, hand-authored FSM — small state count (~10), deterministic transitions	N/A — this is exactly the regime where a real FSM works: few states, no combinatorial blow-up
Meso (§3)	Given the chief complaint, which structured template applies?	A classifier + template library (SOCRATES, others)	The branching depends on <i>content</i> (which complaint), not just position — a template registry scales better than encoding every complaint as FSM states
Micro (§4)	Within a template, what's the next question, and how	Slot-filling frame + guarded LLM I/O	The "state" here is the entire accumulated set of answers, not a

do we fill slots from natural
speech?

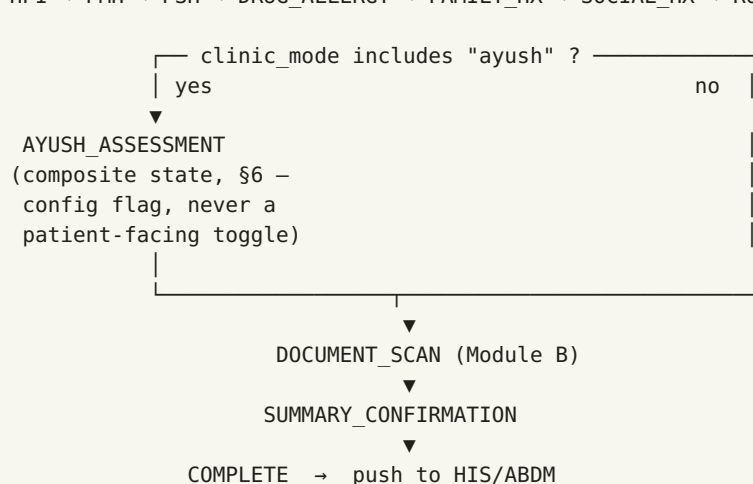
discrete position —
modeling every
combination as FSM
states is
combinatorially
infeasible

This maps directly onto your instinct: yes, a macro FSM for section-level structure is exactly right, and it's the piece the prior draft never actually specified (it drew "State Machine" as one box in a diagram with no state list). The nuance is that the FSM should stop at the macro level — trying to push it down into question-by-question branching within a section is where it breaks.

2 The macro-FSM — Macleod-derived section states

Macleod's Clinical Examination — the standard reference for how a structured history is organized — sequences a history as: presenting complaint → history of presenting complaint (pain, associated symptoms, effects on lifestyle) → past history → drug history (including allergies) → family history → social history/lifestyle → systematic enquiry (review of systems). This is already what v1's section list (CC, HPI, PMH, PSH, DH, FH, SH, ROS) approximates — the useful step is formalizing it as an explicit state graph with real transition and interrupt logic, since that's what actually governs the patient-facing flow.

INIT → CHIEF_COMPLAINT → HPI → PMH → PSH → DRUG_ALLERGY → FAMILY_HX → SOCIAL_HX → ROS



INIT covers language, ABHA auth, consent. CHIEF_COMPLAINT feeds the meso-layer template selection (§3). PMH/PSH/DRUG_ALLERGY/FAMILY_HX/SOCIAL_HX/ROS each load a checklist + voice-elaboration template appropriate to that section.

Global interrupts, available from every state:

- [red flag detected] → REDFLAG_INTERRUPT → EMERGENCY_PROTOCOL (bypass queue, page triage)
- [patient taps Help] → STAFF_ASSIST (pause; resume the same state on return)
- [patient taps Back] → previous state (re-open its slot set for edits)

Transition rule

Advance to the next state when the active meso-template (§3) reports all required slots filled or the patient explicitly signals "nothing else" on an optional slot. Each state can also be entered with some slots pre-filled from document extraction (§7) or a prior ABHA record — the FSM skips asking anything already known with high confidence, and instead runs a lightweight confirm-only pass ("Our records show you're on Amlodipine — still taking it?").

Why these are safe as a literal FSM

Ten-ish states, transitions are almost entirely linear-with-skips, and every transition is triggered by one condition (template complete / explicit skip / interrupt). There is no per-question branching modeled here — that's deliberately pushed down to §3/§4. This is the difference between a maintainable FSM and the combinatorial one that breaks.

3 The meso layer — complaint-driven templates inside HPI

When the macro-FSM enters **HPI**, it hands off to a template selector. The chief complaint (from CC, spoken or tapped) is classified into a category, and that category loads an ordered slot template. This is where your acronym instinct plugs in — SOCRATES isn't a special case, it's one entry in a template registry:

Complaint category	Template	Slots (ordered by clinical priority)
Pain (any site)	SOCRATES	Site → Onset → Character → Radiation → Associations → Timing → Exacerbating/Relieving → Severity
General / constitutional symptom (fatigue, weight loss, weakness)	DOPS (your template — Duration, Onset, Progression, Severity)	Duration → Onset → Progression → Severity, extended with a small associations checklist per symptom
Fever	Fever-specific	Duration → Pattern (continuous/intermittent) → Peak temp → Chills/rigors → Associated symptoms → Travel/exposure correlation
Respiratory (cough, breathlessness)	Respiratory	Duration → Sputum (yes/no, color) → Hemoptysis → Breathlessness grade → Wheeze → Chest pain co-occurrence
GI (pain, bowel change)	GI	SOCRATES core + bowel habit → blood in stool/vomit → appetite → weight change

ON YOUR ACRONYM CHOICE

DOPS is a fine, minimal general-symptom template — the point of this architecture is that it's just another row in the registry, not a special code path. Two named alternatives worth benchmarking it against as you build out coverage: **OPQRST** (Onset, Provocation/Palliation, Quality, Region/Radiation, Severity, Time) and **OLD CARTS** (Onset, Location, Duration, Character, Aggravating, Relieving, Timing, Severity) — both are more widely taught than a from-scratch acronym, which matters when you eventually want physicians to trust the summary without re-deriving your naming scheme. Worth a quick internal comparison on real transcripts before you commit.

Classifying the chief complaint into a category is an LLM/embedding classification call, not a keyword match — patients say "seene mein dard" or "my chest hurts and I feel weak" and the classifier has to map free text (in any of the supported languages) onto one of a fixed, small category list. This is a narrow, well-bounded LLM task (classification into a closed set, not open generation) and is exactly the kind of thing that should be unit-tested against a labeled vignette set before launch, the same way adaptive symptom-checkers are validated against standardized vignette batteries before release.

4 The micro layer — the actual dynamic-questioning mechanism

This is the piece you flagged as the real problem, so here is the turn-by-turn mechanics in full.

TURN LOOP (repeats until the active template's required slots are filled)

1. Dialogue Manager → next_slot = highest unfilled-slot priority in the active template (red-flag-relevant slots weighted first – e.g. for chest pain, radiation + dyspnea are asked before exacerbating/relieving factors)
2. Dialogue Manager → emits a STRUCTURED instruction {slot_id, options, prior_context} – never raw text
3. LLM (constrained) → phrases + translates into the patient's language – output schema allows ONLY question text + a ≤1-sentence acknowledgement of the prior answer
4. TTS + Touch UI → speaks the question; renders synced option buttons drawn from the SAME slot spec (voice and touch can never show/say different things)
5. Patient → responds by voice and/or tap
6. ASR (if voice) → transcript
7. LLM (extraction) → fills the asked slot AND any other open slots the patient mentioned opportunistically in the same utterance (this is what makes it feel dynamic, not the LLM choosing what to ask)
8. Safety Watchdog → re-scans the FULL slot set for red-flag combinations – runs every turn, independent of steps 1–7 (see §9)
9. Confirm-back → ONLY if the slot is red-flag-relevant OR extraction confidence < threshold – not after every single answer (see Appendix A, item 6)
10. Loop to step 1, or transition the macro-FSM (§2) if the template's required slots are all filled or explicitly skipped

The guardrail that makes step 3 and step 7 safe

The LLM is never given a free-text box to decide what to ask or how to interpret an answer — it is bound to a fixed input/output contract at every call. This is the concrete mechanism, not a policy statement:

```
// Dialogue Manager → LLM (phrasing request, step 2/3)
{
  "action": "ask_slot",
  "slot_id": "hpi.radiation",
  "slot_prompt": "Does the pain spread anywhere else?",
  "options": ["No spread", "Left arm", "Jaw", "Back", "Other"],
  "language": "hi-IN",
  "prior_context": {"chief_complaint": "chest pain", "onset": "yesterday morning"}
}
// Allowed LLM output – nothing else is accepted by the schema:
{
  "spoken_text": "...",
  "ack_of_prior_answer": "..." // optional, max 1 sentence
}
```

```
// Patient utterance → LLM (extraction request, step 7)
{
  "action": "extract_slots",
  "template": "SOCRATES",
  "open_slots": ["radiation", "associations", "timing", "severity"],
  "transcript": "yeah it goes into my left arm and i feel a bit breathless"
}
// Allowed LLM output:
{
  "filled_slots": [
    {"slot_id": "radiation", "value": "left_arm",
     "confidence": 0.94, "source": "direct"},
    {"slot_id": "associations.dyspnea", "value": true,
     "confidence": 0.81, "source": "opportunistic"}
  ],
  "unrecognized_mention": null // free text goes here if it doesn't map to
                              // any open slot – logged for the physician,
                              // never silently discarded or force-fit
}
```

WHERE V1'S "LLM ROUTER" BOX NEEDED THIS

The prior draft's dialogue-manager diagram had a "LLM Router (Adaptive)" box producing a "Generative Follow-up (LLM-based, constrained)" branch, but never specified the constraint. Without the schema above, "LLM decides when to generate a follow-up" is exactly the free-running behavior that under-elicits information in practice. The schema is the actual deliverable that box was gesturing at.

Handling the genuinely novel case

When `unrecognized_mention` comes back non-null — the patient said something the template didn't anticipate — the dialogue manager may issue exactly one bounded clarifying question (same schema as above, with `action: "clarify"`), never a diagnostic or advisory statement. The raw mention is always preserved verbatim in a `clinician_note` field on the patient record (§8) regardless of whether it's resolved, so nothing volunteered by the patient is ever silently dropped.

5 Runtime integration — wiring STT, TTS, the LLM, and the FSM together

Four moving parts, four different latency/placement requirements. The mistake to avoid is treating this as one service — each component should be independently swappable and independently scaled.

Component	Runs where	Why
Voice orchestration VAD, turn-taking, barge-in, audio streaming	Kiosk edge (thin layer)	This is generic real-time plumbing — use an existing voice-agent framework (LiveKit Agents or Pipecat) rather than hand-rolling it; both provide streaming VAD, interruption handling, and overlapped stage execution out of the box, which is what gets you a sub-second, non-robotic turn cadence
ASR / TTS	Hospital-local server <i>or</i> cloud API — decision needed, see callout below	Bhashini's ASR/TTS is API-based (MeitY-hosted); AI4Bharat's IndicConformer/IndicTTS are open-weight and self-hostable
Dialogue Manager macro-FSM + meso templates + micro slot-filling + safety watchdog	Hospital-local server (private cloud/on-prem)	This is a fast rule/lookup service (milliseconds), not a generation step — it belongs with the rest of the "heavy AI" tier in v1's hybrid deployment model, on the hospital's own network, not on the literal kiosk box
LLM (phrasing / extraction)	Hospital-local GPU server	Called every turn — needs to be fast and cheap. Can be a smaller/distilled model than the one used for the once-per-visit Module C summary, since its job here is narrow (schema-bound phrasing and extraction, not open generation)

FLAG FROM V1 THAT NEEDS A DECISION, NOT A DEFAULT

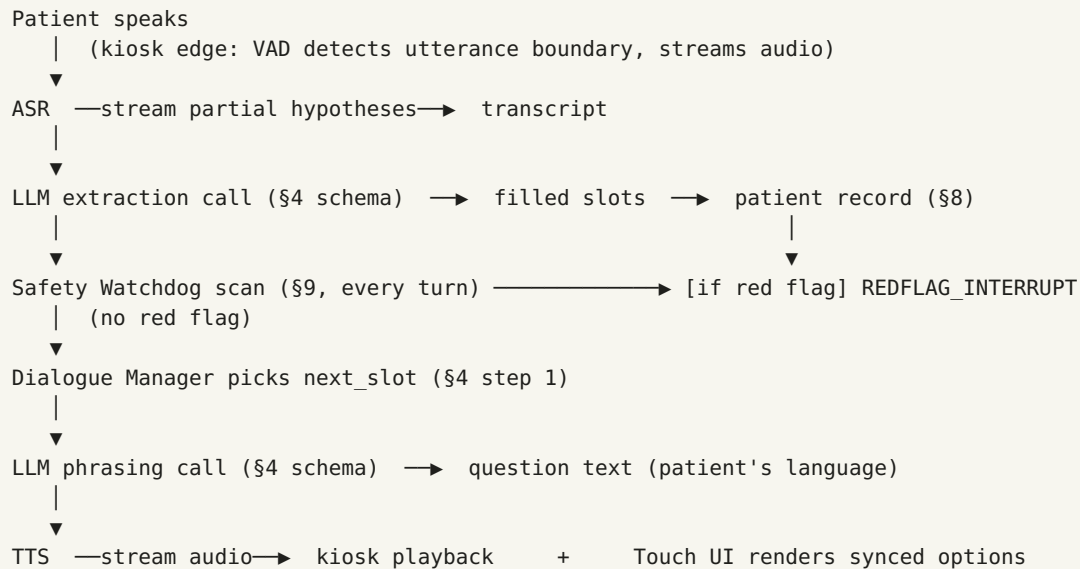
The prior draft states both "Bhashini is the mandatory primary choice" for ASR/TTS *and* "no patient data leaves the hospital network" as a DPDP compliance point. These are in tension: Bhashini's APIs are hosted by MeitY, so raw voice audio sent to Bhashini leaves the hospital's private network. Neither choice is wrong, but the document needs to pick one and be explicit about it in the consent language:

- **Option A — Bhashini as primary:** keep the "government-aligned, free, 22-language" advantages, but explicitly disclose in the consent flow that voice audio is processed by the national Bhashini platform, and confirm Bhashini's data-handling terms satisfy your DPDP Data Fiduciary obligations before launch.
- **Option B — self-hosted AI4Bharat models as primary:** IndicConformer (ASR) and Indic Parler-TTS run on your own GPU server, so the "no data leaves the network" claim becomes literally true; Bhashini becomes a fallback for languages you haven't self-hosted

yet. More infrastructure to own, but a cleaner compliance story for a first government-hospital deployment.

Recommend B for the pilot, A as the scale-out fallback once you have a signed data-processing agreement in place.

ONE TURN, END TO END



Design toward the industry-standard conversational latency budget (roughly 300 ms perceived response time is the baseline modern voice-agent frameworks target) as a north star, but treat it as something to *measure* on your actual hospital network and Bhashini/self-hosted ASR round-trip — not something to assume. Streaming partial ASR results into the extraction step before the patient finishes speaking, and streaming TTS audio out before the full sentence is synthesized, is what gets overlapped-stage latency down; running each stage sequentially and waiting for full completion at each step is the most common way teams accidentally build a system that feels laggy.

6 AYUSH mode as a gated configuration

Your requirement — available only for AYUSH clinics — should not be implemented as a button the patient can tap. Ayurvedic constitutional assessment is only clinically meaningful when ordered by an Ayurveda physician for that department's workflow; letting a patient opt into it themselves invites confusion, wasted interview time, and a summary the receiving physician didn't ask for.

Decision: mode is derived from the encounter context, not selected by the patient

`clinic_mode` is a property of the **session**, resolved at STEP 1 (identification) from one of, in priority order:

1. **Appointment/token record** — if the patient checked in against an OPD department code fetched from the HIS (e.g. "AYUSH-OPD-03"), the kiosk resolves `clinic_mode` from a `department_config` lookup table automatically. This is the primary path for hospitals where kiosks are shared across departments.
2. **Kiosk-level default** — a kiosk physically deployed inside an Ayurveda OPD wing can carry a fixed default in its device config, for hospitals that dedicate hardware per department.
3. **Staff-assisted override** — for walk-ins with no prior appointment record, front-desk staff select the department at token issuance (already in v1's staff-assist fallback for other cases); this, not the patient, sets the flag.

The patient never sees a mode toggle. If a hospital genuinely wants patient-initiated integrative care (both systems, patient's choice), that's a product decision to make deliberately later, gated behind physician sign-off on the intake protocol — not a default.

What actually belongs inside AYUSH_ASSESSMENT — and what doesn't

v1 lists both Dashavidha Pariksha and Ashtavidha Pariksha under Module A (patient-facing conversation). Half of that is right and half of it is a clinical category error worth catching before it's built:

AYUSH_ASSESSMENT (composite macro-state, only loaded when `clinic_mode` includes "ayush")

Dashavidha Pariksha — PATIENT-REPORTABLE, belongs in the conversational engine:

└ Prakriti	constitution — Vata/Pitta/Kapha questionnaire (structured, self-report)
└ Vikriti	current dosha imbalance
└ Sara	tissue quality — self-reported proxy questions
└ Samhanana	body build — partly self-report, partly camera/manual anthropometry
└ Pramana	body measurements — via kiosk camera/manual entry, NOT spoken history
└ Satmya	environmental adaptability
└ Sattva	psychological strength
└ Ahara Shakti	digestive capacity / Agni — self-reportable
└ Vyayama Shakti	exercise tolerance — self-reportable
└ Vaya	chronological vs biological age

Ashtavidha Pariksha — NOT patient-reportable, should NOT be in the kiosk interview:

- x Nadi (pulse) – requires a physician's finger on the patient's wrist
- x Jihva (tongue) – requires direct visual examination by the physician
- x Sparsha (touch) – requires physician palpation
- x Drik (vision/gaze) – requires physician observation
- x Shabda (voice quality), Akriti (general appearance) – observational, physician-performed
- Route these to a physician-facing structured EXAMINATION FORM in Module C / the consultation screen, filled in by the doctor during the visit – not asked of the patient at the kiosk.

WHY THIS MATTERS BEYOND CORRECTNESS

Putting "does your pulse feel weak?" or "describe your tongue" to a patient at a kiosk produces noisy, clinically useless self-report data — patients can't reliably assess their own pulse quality — and it also burns interview time on questions that will simply be re-done by the physician anyway. Removing Ashtavidha Pariksha from Module A shortens the AYUSH interview and makes the summary more trustworthy, not less comprehensive, since that data still gets captured — just in the right place, by the right person.

Sara through Vaya are still a meaningfully large questionnaire (roughly 40–60 discrete data points across the ten categories in typical Ayurvedic intake protocols). Model each of the ten as its own sub-template inside the composite state, using the exact same slot-filling mechanism from §4 — this is not a new engine, it's the micro layer applied to a different, larger template registry, populated with an Ayurvedic physician's input rather than a symptom-branching one.

7 Document extraction — what to keep, what to simplify

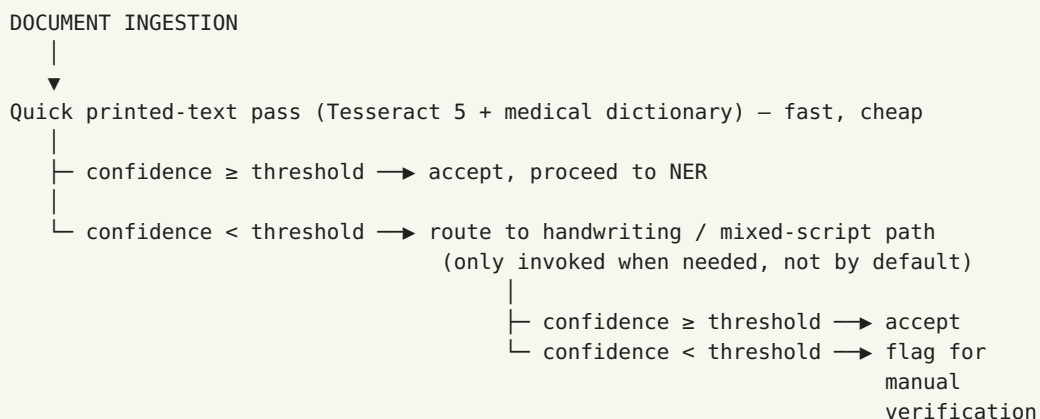
v1's instinct on OCR architecture is directionally correct and worth keeping: route printed text, handwritten text, and mixed-script documents through different specialized paths, and always flag low-confidence extractions for manual verification rather than let the system guess. Two changes are worth making before this becomes a build spec.

DON'T SHIP THE V1 BENCHMARK TABLE AS-IS

The precision figures quoted (98.7% CER for Tesseract 5, 89.6% accuracy for a custom CRNN "trained on IAM + Prescription Corpus," 94.2% CER for the hybrid ensemble, 32.6% exact match for a named VLM) are presented without a traceable source, and the draft itself is machine-generated research ("Generated by Kimi" appears on every page). One specific thing worth checking before repeating any of these: IAM is an English-language Latin-script handwriting dataset — its relevance to Hindi/regional-script handwritten prescriptions is not obvious and isn't explained. Don't put these numbers in front of a hospital review board, a grant committee, or investors until you've either traced them to a real published benchmark or replaced them with numbers you measured yourself against a real sample of Indian OPD prescriptions.

Simplify the routing for an MVP

v1's pipeline always runs three parallel classifiers (document type, language, content type) followed by three parallel OCR paths and a weighted ensemble vote on every document. That's the right end-state, but it's latency- and cost-wasteful to run unconditionally from day one. A confidence-gated cascade gets most of the value at a fraction of the compute:



Escalate to the full parallel ensemble later, once you have real volume and can justify the added infrastructure cost with measured accuracy gains — not as the MVP default.

Merging into one record

Document extractions and conversational slots write into the *same* patient record (§8) with a provenance tag on every value. This is what makes v1's contradiction-flagging idea ("document says diabetic, patient denied in conversation") actually implementable — and it should be extended one step further: contradictions are surfaced to the physician as a flagged diff on the consultation screen, never silently auto-resolved by the LLM. The LLM does not get to decide which source is "true."

8 The unified patient-record schema

One object, written to by the macro-FSM, the meso/micro layers, the document pipeline, and read by the summary generator (Module C). Every value carries its own provenance and confidence — this is the backbone that makes §4's extraction, §6's AYUSH extension, and §7's OCR merge all consistent with each other.

```
{
  "session_id": "sess_8f2a...",
  "clinic_mode": "ayush",           // §6 – "allopathic" | "ayush" | "integrative"
  "macro_state": "HPI",
  "language": "hi-IN",

  "sections": {
    "chief_complaint": {
      "value": "chest pain",
      "category": "pain",           // drives meso-template selection, §3
      "confidence": 0.97, "source": "conversation"
    },
    "hpi": {
      "template": "SOCRATES",
      "slots": {
        "site":      {"value": "chest",      "confidence": 0.95, "source": "direct"},
        "onset":     {"value": "1_day",      "confidence": 0.91, "source": "direct"},
        "radiation": {"value": "left_arm",   "confidence": 0.94, "source": "direct"},
        "dyspnea":   {"value": true,         "confidence": 0.81, "source": "opportunistic"},
        "severity":  {"value": null,         "confidence": null, "source": null}
      }
    },
    "ayush_assessment": {           // present only if clinic_mode includes "ayush"
      "prakriti": { "slots": { "...": "..." } },
      "vikriti": { "slots": { "...": "..." } }
      // ... remaining Dashavidha Pariksha sub-templates, §6
    }
  },

  "document_extractions": [
    {
      "doc_id": "doc_1", "type": "prescription", "ocr_path": "printed",
      "entities": [
        {"field": "pmh.diabetes", "value": true, "confidence": 0.88, "source":
"document:doc_1"}
      ]
    }
  ],

  "contradictions": [
    {"field": "pmh.diabetes", "conversation_value": false, "document_value": true,
      "status": "unresolved_flag_for_physician"}
  ],

  "red_flags": [],                // populated by the safety watchdog, §9
  "clinician_notes": []           // unrecognized_mention text from §4, verbatim
}
```

9 Red-flag architecture — making the three layers actually agree

v1's "rule-based + ML + LLM triple layer" is the right shape but doesn't specify what happens when the layers disagree, which is the only question that matters for a safety system. The fix is a strict authority order, not three equal votes:

Layer	Role	Authority
Rule-based keyword/threshold matching, e.g. chest pain + dyspnea + severity ≥ 7	Runs every single turn, on the full accumulated slot set, without exception	Gating — this layer alone can trigger REDFLAG_INTERRUPT . It is deterministic and auditable, which is what you can defend to a hospital's clinical safety committee.
ML severity scorer fine-tuned clinical NLP model	Adds nuance for borderline cases the rule layer doesn't catch	Refines triage <i>priority ranking</i> among patients already in the queue. Cannot suppress a rule-layer flag, and cannot independently gate an interrupt.
LLM pattern reasoning complex multi-symptom combinations	Catches patterns the fixed rule list didn't anticipate	Must emit an explanation that maps back to a specific rule/threshold it's proposing to add — never a free-form clinical judgment. Any LLM-raised flag is treated as a <i>candidate</i> rule for the rule layer to also check, escalated to a human reviewer for confirmation before it becomes an auto-trigger, not fired directly to the patient.

DESIGN PRINCIPLE
When any layer disagrees, escalate — take the union of flags, never let a lower-authority layer suppress a higher one. For a hospital emergency-detection system, the acceptable error is a false positive that a triage nurse dismisses in ten seconds, not a false negative that delays care. This is also why the rule layer, not the LLM, has to be the thing standing between a patient having a cardiac event and getting flagged — an LLM call failing or being slow should never be a single point of failure for that decision.

10 Revised build sequence

v1's four-phase, twelve-month plan is reasonable in scope but sequences voice, AYUSH, and OCR complexity alongside the core dialogue logic before that logic has been validated. Recommend validating the macro/meso/micro architecture itself first, isolated from every other source of variability:

Phase	Focus	Why this order
0 — Skeleton before v1's Phase 1	Macro-FSM (§2) + 5–8 chief-complaint templates (§3) in touch-only, text-only mode, one allopathic department, no AYUSH, no voice, no OCR	Validates the clinical branching logic and slot-filling mechanism against real patients without ASR noise, translation quality, or document parsing as confounding variables
1	Add self-hosted AI4Bharat ASR/TTS for the pilot's languages (§5, Option B) + the safety watchdog rule layer (§9)	Voice is the highest-variance component (accents, noise, code-switching) — introduce it once the underlying question logic is already proven
2	Document OCR pipeline (§7, confidence-gated cascade, printed-text path first) + Module C summary generation + FHIR/HIS push	Matches v1's existing FHIR mapping work, which is solid and can proceed largely unchanged
3	AYUSH mode (§6) — Dashavidha Pariksha templates only, department-gated, with an Ayurvedic physician co-designing every slot	No existing vendor ontology to borrow from here — this needs dedicated clinical input time, budget for it explicitly rather than bundling it into a generic "scale & refine" phase
4	Full ABDM production gateway, multilingual expansion, scale-out	Unchanged from v1

A Appendix — claim-by-claim critique of the v1 draft

Claim in v1	Assessment	Recommendation
"State Machine + Clinical Ontology + LLM Router" — 3-box dynamic questioning diagram	Correct high-level shape, but no policy detail: doesn't say how the next question is chosen, how free text fills multiple slots, or how templates differ by complaint.	Replaced with the explicit macro/meso/micro hierarchy and turn-loop mechanics in §2–4, including the exact LLM I/O contract.
Bhashini as "mandatory primary" ASR/TTS, alongside "no patient data leaves the hospital network"	Internally inconsistent — Bhashini is a MeitY-hosted cloud API; using it means voice audio leaves the hospital network.	Pick explicitly (§5 callout): self-host AI4Bharat models for the pilot (true "no data leaves"), or keep Bhashini and disclose the data flow in consent language.
OCR benchmark table: 98.7% CER Tesseract, 89.6% acc custom CRNN on "IAM + Prescription Corpus," 32.6% exact match GLM-OCR	No traceable source given; IAM is an English/Latin-script dataset of unclear relevance to Hindi/regional handwriting. Reads as generated placeholder data.	Do not present externally until sourced or replaced with your own measured pilot numbers (§7).
"JMIR 2026 study" ROUGE-1/BERTScore figures for Llama-3-8B vs. Meditron-7B	Same concern as above — precise figures without a verifiable citation trail in the document.	Verify the primary paper and re-quote exactly, or state only the qualitative conclusion (open-weight models are viable) until verified.
Ashtavidha Pariksha listed under Module A (patient-facing conversation)	Clinically incorrect for most sub-items — Nadi, Jihva, Sparsha, Drik require direct physician examination; a patient cannot self-report their own pulse quality.	Move to a physician-facing exam form in Module C (§6). Keep only Dashavidha Pariksha in the patient-facing engine.
"TTS repeats back what was understood" after every input	Good for trust, but doing it after every slot across an 8–12-section interview conflicts with the doc's own 5–8 minute target and will feel tedious.	Confirm-back only for red-flag-relevant or low-confidence slots (§4, step 9); silently accept high-confidence routine answers.
Red-flag "Rule + ML + LLM triple layer" with no disagreement policy	Right components, no authority order — doesn't say what happens when layers disagree, which is the only question that matters for safety.	Rule layer is the sole gate for interrupts; ML/LLM only refine priority ranking (§9).
	Rasa targets open-domain chatbot intent/slot-filling; it's a	Build the macro-FSM and meso-templates as a lightweight custom

*Dialogue manager tech:
"Python + Rasa / Custom
State Machine"*

heavy dependency for a hand-authored, auditable medical ontology that needs to be fully inspectable for clinical safety review.

service you fully control (\$2-4), not a general chatbot framework.

*Document upload always
runs 3 parallel classifiers
+ 3 parallel OCR paths +
ensemble vote*

Right end-state, but latency/cost-wasteful as an MVP default when most documents are cleanly printed.

Confidence-gated cascade for the pilot (\$7); escalate to the full parallel ensemble once volume justifies it.

This document is a design reference to guide implementation planning. It has not been reviewed by a clinician, a DPDP/legal counsel, or an ABDM-certification body, and none of the specific performance figures, timelines, or vendor claims in it should be treated as validated until confirmed against primary sources or your own pilot data.